LifeDocs Press

First Rupee Playbook

The fastest honest path to first income

Skill to income. No fantasy.

A LifeDocs Press Publication

First Rupee

The core playbook for first revenue.



First Rupee — The Fastest Honest Path to Income

🛡️ Honest survival plan. No hype — you were burned by the AI-hype story once and I won't sell you a second one. Every timeline here is real. The goal is your **first paid rupee in 4 months**, then a repeatable trickle, not a fantasy jackpot.

The verdict (read this first)

Quickest path = paid client work for AI video reels — sold directly to local businesses and indie musicians, plus a Fiverr listing running in parallel.

Not ad revenue. Not going viral. Those take *months* and depend on an algorithm you don't control. A paying client depends only on you making one good sample and reaching out to enough people. That's the fastest honest route from ₹0 to ₹1.

Realistic first rupee: 3–14 days if you do the outreach. Not tonight, not a fantasy — but real and close.

All your options, ranked by speed to first rupee

Option	Speed to ₹1	Verdict
Client reels — local business + musicians (direct outreach)	3–14 days	⭐ FASTEST. Do this first.
Fiverr / Upwork / Freelancer.in gig	1–4 weeks	⭐ Run in parallel — set up once, leads trickle in.
Wedding / event / real-estate promo reels (local)	1–3 weeks	Strong cash market near you.

Option	Speed to ₹1	Verdict
Faceless YouTube Shorts / Reels (ad revenue)	2–6 months	Slow. Build later, not for survival.
The tribute film / Suffocating Distance	Not direct income	Portfolio proof — the thing that *lands* the clients.
FRONTLINE LANGUAGE short / LinkedIn posts	Amplifier	Produce once the ramp is already running (your own note).

Your unfair advantages (why this is realistic for *you*)

- **You already pay for Google AI Pro (Flow/Veo).** Your production tool is bought. Most beginners don't have this.
- **You can hold character consistency across shots.** You proved it today — Meher and Neel, same faces across a whole sequence. *Most people flailing with these tools cannot do this.* It is the #1 thing clients actually pay for.
- **You have a locked, cinematic grade and real story instinct.** Your reels look like film, not like AI slop. That's the differentiator.
- **Your emotional/devotional/spiritual register performs in India** — a genuine niche.

The 72-hour action plan

Day 1 — Build ONE killer sample (½ day)

- Cut a **single 15–30 second reel** from assets you can make fast in Flow. Pick the most universally sellable vibe: a warm, cinematic, emotional mood piece (you already have the library/character look dialed in).
- Add simple text/music. Export vertical (9:16).
- This one reel is your entire sales pitch. It doesn't need to be the whole film.

Day 2 — Set up the storefronts (½ day)

- **Fiverr gig**: title like *"I will create a cinematic AI video reel for your brand or song."* Starter price low to get your first review (e.g. ₹800–1,500), then raise it.

Upload your sample.

- Also create a simple **Instagram/YouTube page** with 2–3 sample reels pinned — this is your portfolio link for direct outreach.

Day 3 — Outreach (the part that actually earns)

- **Message 10 indie musicians** on Instagram (independent singers, small music pages) — offer a lyric/mood video for their next release. Attach your sample.
- **Message 10 local businesses** — cafés, boutiques, gyms, real-estate agents, wedding photographers near you. "I make cinematic promo reels — here's a sample — ₹X for one reel."
- **Post once on LinkedIn** with your sample and a line about what you do.

> Volume is everything. 20 messages → 1–2 replies → 1 first client. That's the math. The first rupee is on the other side of outreach, not on the other side of a perfect film.

Pricing (India, starting out)

- First 1–2 jobs (to get reviews/testimonials): **₹800–1,500** per reel
- After a few reviews: **₹2,000–5,000** per reel
- Music videos / multi-shot pieces: **₹5,000–15,000**
- Local business monthly retainer (3–4 reels/month): **₹6,000–15,000/month** ← the real stability goal

---

What NOT to count on (honest guardrails)

- ❌ Don't wait for the full tribute film to be done before earning. The sample is enough.
- ❌ Don't count on YouTube/Reels ad money for survival — it's months away.
- ❌ Don't spend your Flow credits rendering endlessly. Render *just enough* to sell, then let clients' money fund the rest.

- ❌ Don't do the FRONTLINE short yet — your own note says it's an amplifier for *after* the ramp is running.

The one sentence to remember

The film is the proof. The client is the rupee. Outreach is the bridge between them. You already have the hardest part — the skill. What stands between you and your first rupee is 20 messages, not another masterpiece.

Sell Today

Direct-response selling material.



💸 Sell Today — LinkedIn Cash Kit

⚡ Fast money = your existing LinkedIn audience + a direct offer + UPI up front. Post Option 1 now. Reply to every comment within minutes. Have your Drive link + UPI ID ready before you post. And do NOT send the bundle until you've removed the Solana wallet key (see bottom).

Option 1 — The ₹2,999 bundle post (main)

```
I built 6 working AI systems over the last year.
Today, I'm giving away full ownership of all of them. Once. ₹2,999. Yours forever.

This isn't a course or a PDF of theory. It's real, running code + a step-by-step guide:

🎬 An AI that turns a written story into a finished, scored film
🤖 A system that plans, writes, tests & ships its own code
🎥 A faceless-content video pipeline
🛡️ An on-device content-moderation model
📈 A Solana trading study with full source
🧠 A thinking + code-review framework

The part that makes this different: full white-label rights.
Rebrand it. Use it. Resell it and keep 100%. No royalties. No catch.

Sold separately these would run ~₹5,500. Together, today: ₹2,999, one-time.

I'm keeping this to the first 20 people.

👉 Comment "KIT" and I'll DM you the link.

— Kunal
```

Option 2 — ₹499 fast-entry post (lowest friction, fastest yes)

🏃 Use this if you need the quickest possible sale. A ₹499 price is an instant yes for a warm audience — then upsell buyers to the full bundle in DMs.

```
Everyone's talking about AI. Almost no one has the actual tools.
I built 6 of them. This week, you can grab one for ₹499.

Pick your weapon:
🎬 AI that turns a story into a finished film
🎥 A faceless-content video engine
🤖 A self-coding AI dev system
🛡️ A content-moderation model
📈 A Solana trading study (full source)
🧠 A thinking + code-review framework

Any ONE: ₹499. All SIX + full resale rights: ₹2,999.
Real code + a plain-English setup guide. Yours forever.

Comment "AI" and I'll DM you the list. 👇

— Kunal
```

Pin this as your first comment

📌 The first comment boosts reach and tells people exactly what to do next.

```
Grabbing the most interest first 👇 Comment which one you want (or "ALL 6") and I'll send
the link + UPI. First buyers today get priority setup help.
```

DM close scripts (send the second they comment)

For the full bundle:

```
Hey [name]! Here's the full 6-project bundle + setup guide — lifetime access, full resale rights: [Drive link]. It's ₹2,999 via UPI to [your UPI ID]. Send me the payment screensho t and I'll unlock your access right away. 🙌
```

For the ₹499 single project:

```
Hey [name]! Great pick. Here's [project name] — full code + a plain-English setup guide: [Drive link]. It's ₹499 via UPI to [your UPI ID]. Send the screenshot and I'll share it i nstantly. (Want all 6 + resale rights instead? Just ₹2,999.) 🙌
```

Set up before you post (5 minutes)

- [ ] Your UPI ID copied and ready
- [ ] The Drive link ready — but ONLY after the security step below
- [ ] Both saved in your phone's notes app for instant paste into DMs

Post rules for maximum reach

- Post as **plain text** — no outbound links in the post itself (LinkedIn suppresses those). Links go in DMs.
- Reply to **every** comment within minutes. Speed = sales.
- Also DM your 15–20 most engaged followers directly with the offer.
- Keep the "first 20" / "this week" urgency — it makes people act now instead of later.

🚨 Security — do NOT skip before sharing any Drive link: the Solana project ships with a live wallet private key + live API keys hardcoded in 7 files. Delete them and rotate the Helius/Birdeye/QuikNode keys first (Section 0 of your Master Setup Guide). Shipping it compromised will cost you more than it earns.

AI Self-Correction Protocol

Process doctrine for honest debugging and iteration.



AI Self-Correction Protocol — Technical Sessions

Paste this at the start of any technical session, or open this page and tell the AI to follow it.

> **Core principle:** The goal is not to stop making mistakes — that is impossible and not the point. The goal is to *notice* the mistake fast, name it honestly, fix it, and make the policy stronger because of it. A mistake that is caught and corrected is a training step. A mistake that is hidden or repeated without acknowledgment is the only real failure.

SELF-CORRECTION PROTOCOL — ENFORCE BEFORE EVERY FIX

Before writing any patch, config change, code edit, or calculation, complete these checks **IN ORDER**. Do not skip. Do not produce output until all relevant checks pass.

CHECK 1 — Trace the primary execution path, not the fallback

- Identify the default/primary branch first. Read it fully before touching any fallback.
- If code has engine A (default) and engine B (fallback): describe A's behavior only.
- Do not let fallback code color the analysis of what runs on success.
- Confirm explicitly: *"I am describing what runs when the primary path succeeds."*

- **Mistake this catches:** Describing LTXV (fallback) as the rendering engine when WAN (primary, `KIT_ENGINE="wan"` default) is what actually runs.

CHECK 2 — Verify all numbers before writing anything

- Any fix involving a threshold, fps, frame count, size, or time: run the exact arithmetic first.
- Show the result explicitly. Only proceed when the math confirms the fix does what is claimed.
- Example: *"interpolate_fps=24, srcfps=16: 24 > 16×1.2=19.2 → TRUE → interpolation STILL RUNS — fix is wrong."*
- **Mistake this catches:** Setting interpolate_fps=24 (wrong) before checking whether 24 > 16×1.2 still triggers the guard.

CHECK 3 — Surgical fixes only. Never disable to fix.

- Identify the one broken step. Fix that step only.
- Never remove, disable, or bypass surrounding steps to work around a slow or broken step.
- Before adding any kill switch (`KIT_FAST_POST=1`, `skip_X=True`): ask *"Am I fixing the problem or hiding it?"*
- **Acceptable:** a guard that skips a step when genuinely unnecessary.
- **Not acceptable:** disabling a step that produces quality output (grade, face super-res, upscale).
- **Mistake this catches:** Adding `KIT_FAST_POST=1` which exits postprocess before color grade, face super-res, and upscale run.

CHECK 4 — On receiving a correction: stop, re-read, then fix

When the user corrects a claim or fix:

1. **STOP.** Do not write a new fix in the same breath.
2. Re-read the exact source lines that were wrong.

3. State explicitly what was wrong and why.
4. Run CHECK 1–3 on the new fix.
5. Only then proceed.

> Never issue a corrected fix simultaneously with acknowledging the correction. Verify first, fix second.

- **Mistake this catches:** Rushing a corrected interpolate_fps patch without re-reading the WAN graph's actual `frame_rate` value first.

CHECK 5 — Read qualifiers. Never use PROVISIONAL values as facts.

- If a variable has a comment like `# PROVISIONAL`, `# REAL value auto-derived`, `# placeholder`, `# unused`, `# estimated` — do not use it in calculations.
- Derive the real value from the authoritative source (parse the YAML, read the graph, measure the output).
- **Mistake this catches:** Using `TOTAL_SHOTS = 435  # PROVISIONAL; REAL total auto-derived from the story below` as if 435 were the real shot count, producing a nonsensical 30-minute duration for a 2-minute film.

CHECK 6 — Cross-validate numbers against known outputs

- After any calculation, check the result against at least one known ground truth.
- If the result contradicts a known fact, the calculation is wrong — find the error before presenting it.
- Formula: *calculated result ↔ known output → if they don't agree, one is wrong.*
- Example checks:
  - Shot count × avg duration = film duration? `435 × 4s = 1740s ≠ 2 min` → shot count is wrong.
  - GPU parallelism applied? `435 shots ÷ 2 GPUs = 218 per GPU` → use 218 for wall-clock time, not 435.

- **Mistake this catches:** Presenting 435 × 4s = 30 minutes when the film is 2 minutes; using 435 (not 218) after applying "÷2 GPUs".

CHECK 7 — Verify actual active hardware from runtime output, not UI labels

- Kaggle UI may show "GPU T4 ×2" but the notebook's `_safe_by_ram` / `_ngpu` check caps active GPUs based on VRAM and system RAM.
- Never assume parallelism from the UI. Read the printed `[gpu]` log line or the `_ngpu` calculation before stating how many GPUs are active.
- **Mistake this catches:** Assuming dual-T4 parallel rendering when the RAM check limits execution to 1 GPU.

CHECK 8 — Read content before advising on it. Never assume.

- Before telling a user to add, change, or fix something: read the actual current content to confirm it isn't already there.
- This applies especially to your own previously delivered files — do not assume their state from memory.
- **Mistake this catches:** Advising the user to add `interpolate_fps=16` to a notebook that already contained that exact line because I delivered it.

All mistakes from this session — mapped to checks

Mistake	Check
Described LTXV (fallback) as the primary rendering engine	CHECK 1
Set interpolate_fps=24 without verifying 24 > 16×1.2	CHECK 2
Added KIT_FAST_POST=1, disabling grade + face super-res + upscale	CHECK 3
Rushed a corrected fps patch without re-reading the WAN graph	CHECK 4
Used TOTAL_SHOTS=435 marked PROVISIONAL as a real value	CHECK 5
435 shots × 4s = 30 min contradicts known 2-min film duration	CHECK 6
Used 435 instead of 218 after stating dual-T4 halves work	CHECK 6

Mistake	Check
Assumed dual-T4 parallel render; actually only 1 T4 active	CHECK 7
Told user to add a line already present in my delivered notebook	CHECK 8

How to activate

At the start of a session, say:

> *"Follow the self-correction protocol at [this page]."*

Or paste the eight checks directly into the first message of a new thread.

Before every tool call that produces output: confirm which checks apply and that they are done.

🧠 REINFORCEMENT LEARNING LAYER

This layer converts the protocol from a static checklist into a self-improving feedback loop. Every interaction is a training step.

RL-1 — Value Estimation (run BEFORE every response)

Before writing any response involving a claim, fix, or calculation, score the planned output on three axes:

Axis	+1	0	-1
Accuracy	Every claim verified from source	Some assumptions	Unverified guesses
Completeness	Fully addresses the request	Partial	Misses the point
Check compliance	All relevant checks passed	Some skipped	Checks ignored

Rule: Proceed only if total score ≥ 2. If score < 2 — do NOT respond yet. Run the failing check first.

RL-2 — Reward Signals

Event	Reward	Action
User accepts without correction	**+1**	Reinforce — note what worked
User corrects a claim or fix	**-1**	Penalty — trigger RL-3 immediately
User asks for clarification	**0**	Uncertainty — state confidence explicitly next time

RL-3 — Policy Update on -1 Reward (mandatory after every correction)

When a correction is received, execute in order:

1. **Name the mistake type precisely** — wrong engine, wrong number, wrong assumption, ignored qualifier
2. **Name which check would have caught it** — CHECK 1 through CHECK 8
3. **If no existing check covers it** — create a new check and add it to this page
4. **Confirm the update:** *"Updated rule: [new rule]. Added as CHECK N."*

Never skip this sequence. A correction without a policy update is wasted signal.

RL-4 — Exploration Rule (strategy switching)

- If the **same mistake type occurs twice**: the current approach is structurally wrong — switch strategy entirely.
- Do **not** retry the same fix pattern that already failed once.
- When genuinely uncertain: say *"I need to verify X before answering"* — never guess and present it as fact.
- Exploration means trying a different verification method, not rephrasing the same wrong answer.

RL-5 — Confidence Calibration (mandatory on all claims)

Before every number, technical assertion, or interpretation:

- State the **source** — file line number, runtime output, user-confirmed fact

- If no source exists: say **"estimated, not verified"** explicitly
- Never present an estimate as a fact
- Confidence vocabulary:
  - *"Confirmed from [source]"* — verified
  - *"Estimated — needs verification"* — uncertain
  - *"I don't know — let me check"* — unknown

RL-6 — Replay Buffer (the mistakes table IS the policy)

The mistakes table below this section is the agent's replay buffer. Every -1 reward adds a row. The table accumulates across sessions.

The longer the table, the stronger the policy.

When starting a new session: re-read the full mistakes table before taking any action.

📋 Replay Buffer — Cumulative Mistake Log

#	Mistake	Check	Rule added/updated
1	Described LTXV (fallback) as primary engine	CHECK 1	Always trace KIT_ENGINE default path first
2	Set interpolate_fps=24 without running math	CHECK 2	Run threshold arithmetic before writing patch
3	Added KIT_FAST_POST=1 — disabled grade+face SR+upscale	CHECK 3	Never disable quality steps to fix timing
4	Rushed corrected fps fix without re-reading WAN graph	CHECK 4	Stop → re-read source → verify → then fix
5	Used TOTAL_SHOTS=435	CHECK 5	Never use variables with PROVISIONAL/placeholder comments

#	Mistake	Check	Rule added/updated
	marked PROVISIONAL as real value		
6	435 shots × 4s = 30 min — contradicts known 2-min film	CHECK 6	Cross-validate all calculations against known outputs
7	Used 435 instead of 218 after stating ÷2 GPU parallelism	CHECK 6	Wall-clock time = shots_per_GPU × time, not total shots
8	Assumed dual-T4 active from UI; only 1 T4 runs	CHECK 7	Read _ngpu runtime log, not Kaggle UI label
9	Told user to add line already present in delivered notebook	CHECK 8	Read content before advising — never assume file state

⚙️ How the RL loop works in practice

```
USER MESSAGE
     ↓
RL-1: Score planned response (accuracy + completeness + check
compliance)
     ↓
  Score ≥ 2?  →  NO  →  Run failing check → revise response
     ↓ YES
Deliver response
     ↓
User reaction?
  +1 (accepted)    →  Note what worked. Reinforce.
   0 (clarify)     →  Add confidence qualifier to that claim
type.
  -1 (correction)  →  RL-3: name mistake → find check → updat
e policy → add to replay buffer
```

```
      ↓
Next message: re-read replay buffer before starting
```

This loop does not stop. Every session makes the policy stronger.

Demo-Ready Playbook

Checklist for making projects sale-ready.

🧪

Demo-Ready Playbook — Battle-Test, Live-Run & Sell Your Other Projects

🧪 A project is **demo-ready** when a stranger can watch it *work* in under two minutes and believe it. Two pillars: **battle-tested** (runs on a clean machine, on inputs you didn't hand-pick, without babysitting) and **live-run** (you can execute it end-to-end right now and produce a real, recorded output).

The demo-ready method (repeat for any project)

1. **Name the ONE claim the demo must prove.** One, not five. (“Detects unsafe images on-device in milliseconds.” “Holds a consistent face on a free GPU.”)
2. **Clean-room run.** Fresh environment (new Kaggle / Colab / venv / phone), from the *exact zip you'd ship*. If it breaks here, every buyer hits the same wall — fix it or document it.
3. **Adversarial inputs.** Feed it data you did *not* cherry-pick. Note where it fails. A disclosed limitation beats a hidden one.
4. **Capture proof.** Screen-record the run and save the artifact — output file, metrics, confusion matrix. Numbers beat adjectives.
5. **Write the honest “does NOT do” list** from what you saw in step 3. This is your refund shield.
6. **Package the exact thing you ran** — not a “cleaned-up” version you never tested.

7. **Cut a sub-90-second proof clip:** the one claim → the live run → the result. That clip is what sells.

Your projects, made demo-ready

Project	Prove this one claim	Battle-tested means	Live run means	Honest limit to disclose
SafeScreen (moderation model + SDK)	Flags unsafe images on-device, offline, fast, with real precision/recall	Eval script on a held-out set you did NOT train on → precision, recall, F1, confusion matrix, false-positive rate on safe images	A tiny demo page: upload an image → flagged in ms, offline — recorded	Per-class weakness; only claim the classes where recall is strong
Solana Sniper (study + source)	A rigorous 6-month paper-trading study + hardened architecture + full source	Re-run the paper-trading / backtest harness; show equity curve, win rate, drawdown honestly (losses included)	A paper run on historical data — reproducible, ZERO real funds, secrets stripped	Not financial advice; not a live trader; past results ≠ future
Convergence Framework (kit)	Decide in an hour if a goal is formalizable — before building for months	Apply it to several real objectives, including a deliberately messy / non-formalizable one	A worked example on video: real goal → run the framework → verdict	It's a thinking discipline, not code that runs; the value is the judgment
GODSHAND film kit (done)	Consistent face + film grade on a free GPU	Clean fresh-Kaggle run from the shipped zip	The 4-shot demo film + walkthrough (already speced)	Needs Kaggle + HF token; ~8–9 min/shot; stylized output

🧭 **Do SafeScreen next.** Per your own roadmap it's the cleanest B2B case — recurring buyers, zero legal/financial risk, and the demo is just an eval table + a 60-second upload-and-flag screen recording.

⚠️ Honesty guardrails

- **api_money_bot — do NOT sell or demo.** It's a ToS-violating key harvester. No demo makes that safe; it's a legal tail you can't carry.
- **Solana Sniper — never demo with real money or pitch it as a profit machine.** Paper-trading study only. That framing is both the honest truth and your legal shield.
- **Strip secrets before zipping** every project — remove .env / .pem / key files, add a .env.example. (You already flagged this for Solana.)
- **Always record on a clean machine** so a buyer's first-run experience equals your demo.

Which agent should do the iterating?

Yes — Antigravity (Google's agentic IDE) is a strong, honest fit for you.

Why it fits: its agent operates across the editor, terminal, and browser autonomously — it writes code, runs it, reads the failure, and fixes it, then re-runs. That is exactly the patch → run → fix loop you don't want to do by hand. During the public preview it's free, with access to Gemini 3 Pro and Claude models — the right price for your situation. Reviewers currently rate it the best *free* agentic IDE.

The honest caveats:

- Rate limits are tight — quota refreshes roughly every 5 hours, and even paying users hit walls. Work in focused sessions; don't expect it to grind unattended for hours.
- Costs past the free quota aren't clearly documented yet.
- It runs on **your** machine, so it can iterate locally on the CPU-friendly projects (SafeScreen eval, Convergence, the Solana paper harness). Anything GPU-heavy (the film) still belongs on Kaggle.

If it throttles you, fall back to:

- Cursor (free tier) — polished, same kind of loop.
- Claude Code — predictable flat pricing, excellent at whole-repo work.
- aider or OpenCode / Codex CLI with your own API key — unlimited, most frugal.

Bottom line: start with Antigravity free on SafeScreen. If rate limits interrupt you mid-loop, switch to Cursor free or aider.

Ready-to-paste agent prompts

Paste the **master prompt** first, then paste the matching **project block** right under it in the same message.

Master prompt (use for every project)

```
You are my senior engineer. Your goal: make this project DEMO-READY, which means battle-t
ested + live-run.

Context: I sell small digital products. This must run for a stranger on a clean machine a
nd produce ONE believable proof in under 2 minutes.

The ONE claim this demo must prove: <FILL IN>

Work this loop autonomously until it is done. Do NOT stop at the first error to ask me —
keep iterating:
1. Create a fresh virtual environment and install from the exact files in this project. L
ist every dependency you install.
2. Run the entry point / test / eval command. If it fails, read the real error, fix the c
ode, and re-run. Repeat until it runs clean.
3. Feed it inputs I did NOT cherry-pick (edge cases, empty, malformed, oversized). Log ex
actly what breaks.
4. Produce a real artifact as proof: an output file plus metrics (numbers, not adjective
s). Save it to disk.
5. Write an honest "what it does NOT do" list from the failures you saw in step 3. Do not
hide them.
6. Strip all secrets (.env, .pem, keys, tokens). Add a .env.example. Repackage the exact
tested version into a clean zip.
7. Write a 6-line README: what it is, the one claim, how to run it in 3 steps, known limi
ts.

Hard rules:
- Never fake, mock, or hardcode results. If a number is bad, show the real number.
- Never invent capabilities. Only claim what you actually ran.
```

```
- Print every command you ran so I can reproduce it on camera.
- Finish by giving me a sub-90-second demo script: the claim -> the command -> the resul
t.
```

SafeScreen (moderation model + SDK)

```
[Paste the master prompt above first, then this:]

The ONE claim: "Flags unsafe images (violence / gore / NSFW) on-device, offline, in ~30-5
0ms, with real precision and recall."

Specifics:
- The model is toxic_detector.onnx; run it with onnxruntime on CPU.
- First, confirm the model loads and classifies ONE image end-to-end before anything els
e.
- Build a held-out test set I did NOT train on. Report precision, recall, F1, a confusion
matrix, and the false-positive rate on safe images — PER CLASS.
- Only keep marketing claims for classes where recall is actually strong. Flag weak class
es as limits.
- Live-run demo: a minimal FastAPI or Gradio page — upload an image, get back the flagged
label + confidence, fully offline. Record it.
```

Solana Sniper (paper-trading study + source)

```
[Paste the master prompt above first, then this:]

The ONE claim: "A rigorous 6-month paper-trading STUDY with a hardened architecture and f
ull source — NOT a live money-maker."

Specifics:
- Run ONLY in paper-trading / backtest mode on historical data. NEVER connect real funds,
real keys, or mainnet execution. This is a hard safety rule.
- Re-run the Monte Carlo / backtest harness. Report the equity curve, win rate, and max d
rawdown, and show losing runs honestly.
- Strip every secret and private key. Add .env.example. Verify no wallet or key is left i
n the zip.
- The docs must state, verbatim: not financial advice; paper-tested only; live performanc
e unverified; past results do not predict future results.
- Live-run demo: reproduce the study's output from the shipped zip with a single command.
```

Convergence Framework (kit)

```
[Paste the master prompt above first, then this:]

The ONE claim: "Decide in under an hour whether a goal is formalizable — before you build
for months."

Specifics:
- This is a thinking framework + system prompt, not a long-running program. "Battle-test"
here means: apply it to 3-4 real objectives, including one deliberately messy / non-forma
lizable goal, and show it correctly flags the messy one.
- Produce a worked-example doc for each: real goal -> framework applied -> verdict.
- Package the system prompt + kit + the worked examples together.
- Be honest in the README: the value is the judgment and discipline, not executable code.
```

Where to sell + who buys

Creators — for the film kit

(Full targeting lives on the GODSHAND listing page; the joinable servers:)

- YouTube Faceless (~20k) — huge, but **strictly no self-promo**; give value first.
- Faceless Incubator (~38k) — largest faceless-creator hub.
- Faceless Success — free community, story-sharing friendly.
- Wan (official, ~6.6k) — the exact model your kit uses.
- AI HUB (~21k) — broad create/learn/collaborate server.
- Reddit: r/aitubers, r/aifilmmaking, r/comfyui, r/StableDiffusion, r/PartneredYoutube.

Developers — for SafeScreen, Convergence, Solana study

Different buyers entirely (they don't hang out with faceless YouTubers):

- Reddit: r/androiddev, r/iOSProgramming, r/computervision, r/MachineLearning, r/SideProject; r/algotrading (Solana *study* only — respect their no-promo/no-signal rules).
- **Indie Hackers** and **Hacker News "Show HN"** — ideal for a battle-tested dev tool with real metrics.

- Mobile-ML / computer-vision Discords for SafeScreen; agent-builder communities for Convergence.

India-specific

- Indian AI Production — Telegram + linked Discord, active AI-video crowd.
- **AI Learners India** — Hindi, no-code AI community; friendly to beginners on a budget.
- Heads-up: Facebook "low-budget Indian AI video creator needed" posts (~₹100/video) are *gig-hire* leads, a different market — but they're warm prospects who might buy the kit instead of hiring out.

Selling-platform reality

- **No free discovery.** Gumroad / Payhip won't send you cold traffic — you bring your own from posts and clips.
- **Start price low, scale with proof.** Don't launch high with zero sales.
- **Clean, professional cover images** — don't let it look AI-thrown-together.
- **Fees:** Payhip ~5%, Gumroad ~10%. Factor it in.
- **People buy fixes to small annoying problems**, not grand solutions. Frame each demo around the one irritation it removes.

The film kit is your template. Same seven steps, per project — battle-test, live-run, record the proof, package the exact thing, sell the outcome.

A no-nonsense practical guide for creators, freelancers, and solo builders who have skill but haven't crossed into paid work yet. Packaging, offer design, outreach, and first conversion ? stripped of fantasy.

ISBN-13: 978-0-XXXXXX-XX-X